

Smart Railway Reservation System

Build Your Own Project (BYOP) - VITyarthi

Submitted By: Kartikeya

Program: Computer Science

Date: March 2026

2. Introduction

The Smart Railway Reservation System is a robust, object-oriented application designed to automate train ticketing processes. Developed in Java, the system provides a comprehensive console-based interface for managing passenger profiles, scheduling trains, booking tickets, and handling waitlists. It serves as a practical implementation of fundamental computer science concepts, specifically demonstrating data structures (Collections), enumeration, and multithreading to ensure a secure, race-condition-free booking environment.

3. Problem Statement

Traditional, non-synchronized booking systems frequently encounter critical failures when exposed to concurrent user access. If multiple passengers attempt to reserve the last available seat on a train simultaneously, the system may register multiple confirmed bookings for a single resource. Furthermore, managing cancellations manually can lead to inefficiencies where seats remain empty despite a backlog of waiting passengers. The objective of this project is to build a reliable reservation architecture that guarantees thread-safe bookings and fully automates the waitlist promotion pipeline.

4. Functional Requirements

- **Passenger Registration:** The system must accept and store passenger details (ID, Name, Age, Gender, Phone) with validation checks.
- **Train Management:** The system must maintain a catalog of trains with defined routes, total capacities, and real-time seat availability.
- **Ticket Booking:** Users must be able to book available seats. The system should generate a unique alphanumeric PNR for confirmed bookings.
- **Waitlist Queuing:** If a train reaches maximum capacity, subsequent booking requests must be placed in a sequential waitlist.
- **Cancellation & Promotion:** Users can cancel active bookings via their PNR. Upon cancellation, the system must release the seat and immediately promote the first eligible passenger from the waitlist to confirmed status.
- **System Reporting:** The system must generate real-time metrics showing total confirmed bookings, total cancellations, and waitlist counts.

5. Non-functional Requirements

- **Performance:** The application must process bookings and waitlist promotions instantly, leveraging efficient Java Collections (e.g., ArrayList, LinkedList).
- **Reliability (Thread Safety):** The core booking engine must use synchronization to guarantee that concurrent booking requests do not result in overbooking or data corruption.
- **Maintainability:** The codebase must adhere to strict Object-Oriented Programming (OOP) principles, utilizing clear module separation (Service layers, Data models).
- **Usability:** The console interface must provide clear navigation, robust input validation, and informative error handling to guide the user seamlessly.

6. System Architecture

The system is built on a monolithic, three-tier internal architecture tailored for a localized Java environment:

- **Presentation Layer:** The CLI `main` method utilizing `Scanner` for menu-driven user interaction and input validation.
- **Service Layer:** Contains business logic managers (`PassengerService`, `TrainService`, `BookingService`). This layer coordinates interactions between entities, manages Collections, and enforces synchronization locks.
- **Data Model Layer:** Plain Old Java Objects (POJOs) defining the structure of the system's entities (`Passenger`, `Train`, `Booking`, `WaitingRequest`) along with state definitions using Enums (`Gender`, `BookingStatus`).

7. Design Diagrams

Use Case Design Structure

Actor	Primary Use Cases
Passenger / User	Register Profile, View Available Trains, Book Ticket, Cancel Ticket
System / Admin	Manage Waitlist Promotion, Generate System Reports, Handle Concurrent Requests

Class Diagram Representation (Core Modules)

- **Entities:** Passenger (id, name, age), Train (trainNo, totalSeats, occupiedSeats[]), Booking (bookingId, passenger, train, seatNumber).
- **Services:** BookingService (Contains bookings ArrayList, waitingList Queue). Key methods: bookTicket(), cancelBooking(), promoteWaitingPassenger().
- **Threads:** BookingThread (extends Thread, executes parallel booking attempts).

Workflow / Process Flow

1. User requests Booking -> System checks `Train.getAvailableSeats()`.
2. If Seat > 0 -> System marks seat as occupied, generates PNR, status = CONFIRMED.
3. If Seat == 0 -> System creates `WaitingRequest`, adds to Queue, status = WAITING.
4. User requests Cancellation -> System frees seat, finds matching train in Queue.
5. If Queue match found -> System re-allocates freed seat to waitlisted user, generates PNR, status = CONFIRMED.

8. Design Decisions & Rationale

Several architectural decisions were made to optimize system performance and fulfill the project requirements without relying on external databases:

- **In-Memory Collections:** Java's `ArrayList` was chosen for storing Passengers, Trains, and Bookings due to its fast iteration and random access properties. `LinkedList` (acting as a `Queue`) was selected for the Waitlist to ensure strict First-In-First-Out (FIFO) processing.
- **Boolean Arrays for Seat Mapping:** Inside the `Train` class, `boolean[] occupiedSeats` allows for immediate index-based seat allocation and deallocation ($O(n)$ search, $O(1)$ allocation), bypassing the need for complex seat objects.
- **Synchronization Locks:** The `synchronized` keyword is applied directly to `bookTicket` and `cancelBooking` in the `BookingService`. This implicitly uses the service instance as a monitor lock, ensuring only one thread can modify seat availability arrays and lists at a time.

9. Implementation Details

The solution is written in Java 8+ and encapsulated entirely within the `RailwayReservationSystem.java` file. It leverages nested static classes to modularize domain entities. The `UUID.randomUUID()` method is used to generate highly unique, 6-character alphanumeric PNRs. Thread simulation is achieved by extending the `Thread` class in `BookingThread`, which overrides the `run()` method to trigger the `bookingService.bookTicket()` protocol simultaneously for multiple instantiated user profiles.

10. System Execution Results

Below is a representation of the terminal output demonstrating the core functionalities, particularly the concurrency handling:

```
=====
      SMART RAILWAY RESERVATION SYSTEM
=====
1. Register Passenger
4. Book Ticket
9. Concurrent Booking Demo
...
Enter Choice: 9

===== CONCURRENT BOOKING DEMO =====
Two passengers are trying to book the same train...

Booking Successful!
PNR: PNR8B4A12 | Passenger: Aman | Train: Concurrent Express | Seat: 1 | Status:
CONFIRMED
Riya added to Waiting List for Concurrent Express.
Concurrent Booking Demo Completed.
```

```
Enter Choice: 6
----- CANCEL BOOKING -----
Enter PNR: PNR8B4A12
Booking Cancelled Successfully.
Released Seat: 1

Waiting List Passenger Promoted!
PNR: PNRC9F2E7 | Passenger: Riya | Train: Concurrent Express | Seat: 1 | Status:
CONFIRMED
```

11. Testing Approach

Testing was conducted systematically via the interactive command-line interface:

- **Input Validation Testing:** Fed invalid string formats to integer inputs and out-of-bounds phone numbers to verify robust exception handling and regex checks.
- **Functional Testing:** Executed end-to-end user journeys (Register -> Book -> Cancel -> View Reports) to ensure state transitions were accurate.

- **Concurrency Testing:** Built a dedicated "Concurrent Booking Demo" (Option 9) that programmatically spawns two threads competing for a train with exactly 1 total capacity. This definitively proved that the synchronization lock prevents double booking.

12. Challenges Faced

The primary challenge was managing the state of seats during the waitlist promotion sequence. When a cancellation occurs, the released seat number must be immediately captured and re-assigned to the waitlisted user before any parallel thread can claim it. This required careful restructuring of the `cancelBooking` method to ensure the promotion logic remained within the synchronized block, guaranteeing atomicity.

13. Learnings & Key Takeaways

Developing this system provided deep insights into backend service orchestration. Specifically, it clarified the practical applications of Java's Queue interface for FIFO operations. Furthermore, debugging race conditions in the concurrent booking demo cemented a strong understanding of monitor locks and thread lifecycle management in Java.

14. Future Enhancements

- **Database Integration:** Migrate from volatile in-memory collections to a persistent relational database (e.g., MySQL) using JDBC or Hibernate.
- **Payment Gateway Simulation:** Introduce fare calculations, passenger classes (First Class, Sleeper), and simulated transaction processing before confirming a PNR.
- **Graphical Interface:** Develop a frontend using JavaFX or Spring Boot with a React/Angular web client to improve user accessibility.

15. References

- VITYarthi Build Your Own Project Guidelines and Requirements Document.
- Oracle Corporation. *Java Documentation: Collections Framework*.
- Oracle Corporation. *Java Documentation: Concurrency and Multithreading*.